

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..20..22

TROUBLESHOOTING

 Copy as Markdown

Troubleshoot Firebase authentication

This page helps you identify common Firebase authentication failures with Serverpod, explains why they occur, and shows how to resolve them. For issues with Firebase itself, see the [Firebase Auth documentation](#).

Setup checklist

Go through this before investigating a specific error. Most problems come from a missed step.

Firebase Console

- ☐ Create a **Firebase project** in the [Firebase Console](#).
- ☐ Add an app for **every platform you support** (iOS, Android, web) inside the Firebase project. Each platform needs its own registration.
- ☐ Generate a **service account key** from **Project settings** > **Service accounts**.
- ☐ Enable the **authentication methods** you want to use under **Security** > **Authentication** > **Sign-in method**.
- ☐ Confirm **Firebase App Check** is **disabled** (enable it later only after the client integration is in place).
- ☐ In **Authentication** > **Settings** > **Authorized domains**, the list includes `localhost` for development and your production domain when you ship.

Server

- ☐ Paste the service account JSON into `firebaseServiceAccountKey` in `config/passwords.yaml`. See [Store the service account key](#).
- ☐ Confirm the `project_id` inside `firebaseServiceAccountKey` matches the Firebase project the client is using.
- ☐ Add `FirebaseIdpConfigFromPasswords()` to `identityProviderBuilders` in `serverpod_auth.dart`.
- ☐ Create a `FirebaseIdpEndpoint` file in `lib/src/auth/` extending `FirebaseIdpBaseEndpoint`.
- ☐ Start the server with `serverpod start`, then create and apply the migration (press **M**).



Ask AI

Client

- ☐ Install `firebase_core`, `firebase_auth`, and `serverpod_auth_idp_flutter_firebase` in your Flutter project.
- ☐ Run `flutterfire configure` so `firebase_options.dart` is generated.
- ☐ Verify the platform config files are in place: `GoogleService-Info.plist` (iOS, in the Runner target) and `google-services.json` (Android, in `android/app/`).
- ☐ Import `firebase_options.dart` and call `Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform)` before creating the Serverpod client.
- ☐ Call `client.auth.initializeFirebaseSignIn()` after `client.auth.initialize()` inside `initializeClient()` in your Flutter app's `lib/client.dart`.
- ☐ Call `controller.login(user)` after Firebase authentication completes.

Server crashes on first Firebase sign-in with "no such table"

Problem: The server builds and starts, but crashes when a user tries Firebase sign-in. The error cites a missing table (like `serverpod_auth_idp_firebase_account`).

Cause: The database migration that creates the provider's tables was never created or applied.

Resolution: In the running `serverpod start` terminal, press **M** to create and apply the migration.

Token verification fails with "invalid signature"

Problem: The server rejects Firebase ID tokens with a signature verification error.

Cause: The ID token itself failed verification against Google's public certificates. The token was truncated or corrupted in transit, or what was sent is not a Firebase ID token.

A project mismatch produces `Invalid issuer` or `Audience does not match` instead, and broken JSON in `passwords.yaml` fails at startup (see the parse-error entry below).

Resolution:

1. Log the token length in the app before sending it, and confirm the server receives the same value.
2. Confirm the app sends the Firebase **ID token** (`getIdToken()`), not an access token or a custom token.
3. For `Invalid issuer` or `Audience does not match`, verify the `project_id` in your `firebaseServiceAccountKey` matches the project in `firebase_options.dart`.

Token verification fails with "token expired"

Problem: The server rejects Firebase ID tokens with a token expiration error.

Cause: Firebase ID tokens expire after one hour. If the server's system clock is significantly off, valid tokens may appear expired.

Resolution: Check that the server's system clock is accurate. If the client token is genuinely expired (e.g., the user's app was backgrounded for a long time), the client should re-authenticate with Firebase to obtain a fresh ID token before calling `controller.login()`.

Server fails to parse `firebaseServiceAccountKey` from `passwords.yaml`

Problem: The server crashes on startup with a JSON parsing error related to `firebaseServiceAccountKey`.

Cause: The YAML block scalar indentation is incorrect. The `firebaseServiceAccountKey` key uses `|` (literal block scalar), which requires every line of the JSON to be indented at the same level relative to the key.

Resolution: Make sure the JSON block is indented consistently under the `|`:

```
development:
  firebaseServiceAccountKey: |
    {
      "type": "service_account",
      "project_id": "...",
      "private_key": "..."
    }
```

Every line of the JSON must be indented by at least one level more than `firebaseServiceAccountKey`. Mixing tabs and spaces can also cause issues.

`FirebaseAuth.instance.currentUser` is null after sign-in

Problem: After the Firebase sign-in flow completes, `FirebaseAuth.instance.currentUser` returns null, so `controller.login(user)` never gets called.

Cause: `Firebase.initializeApp()` was not called before attempting authentication, or was called with the wrong options.

Resolution: Make sure `Firebase.initializeApp()` is called in your `main()` function before any Firebase operations:

```
await Firebase.initializeApp(
  options: DefaultFirebaseOptions.currentPlatform,
);
```

If you haven't run `flutterfire configure`, do so to generate the `firebase_options.dart` file.

Sign-in succeeds in Firebase but fails to sync with Serverpod

Problem: The user authenticates with Firebase (the Firebase UI shows success), but the Serverpod session is never created. The `onError` callback on `FirebaseAuthController` fires.

Cause: The Firebase ID token could not be verified by the server. Check the server logs first, the error message will narrow this down to one of the following causes:

1. **Missing service account key:** The `firebaseServiceAccountKey` is not present in `passwords.yaml`, or the JSON is invalid.
2. **Missing endpoint:** You did not create the endpoint class extending `FirebaseIdpBaseEndpoint`. Without it, the client has no endpoint to call.
3. **Missing migration:** The provider's database tables don't exist yet. In the `serverpod start` terminal, press **M** to create and apply the migration.
4. **Project mismatch:** The service account key belongs to a different Firebase project than the one configured in your Flutter app. Compare `project_id` in `firebaseServiceAccountKey` against the project in `firebase_options.dart`.
5. **App Check enabled prematurely:** If you enabled Firebase App Check before the client integration is in place, every request will be rejected with an App Check assertion error. Disable App Check until the client is wired up.

Email validation rejects phone-only users

Problem: Users who sign in with phone authentication are rejected. The app receives `FirebaseEmailNotVerifiedException`, or a generic `FirebaseIdTokenVerificationException` when the validator throws a plain exception.

Cause: A custom `firebaseAccountDetailsValidation` callback requires a verified email, but phone-only users don't have an email. The default validation accepts phone-only authentication, so this only happens with a custom validator.

Resolution: Guard on the presence of an email before requiring verification. The built-in `FirebaseIdpConfig.requireVerifiedEmail` does exactly this, or write it yourself:

```
firebaseAccountDetailsValidation: (accountDetails) {  
  if (accountDetails.email != null && accountDetails.verifiedEmail != true) {  
    throw FirebaseEmailNotVerifiedException();  
  }  
},
```

User signed out of Serverpod but still signed in to Firebase

Problem: After calling sign-out on Serverpod, the user's Firebase session remains active. The next time the app opens, Firebase still has an authenticated user.

Cause: `initializeFirebaseSignIn()` was not called during app initialization. This method sets up the automatic sign-out sync between Serverpod and Firebase.

Resolution: Make sure you call `initializeFirebaseSignIn()` after initializing the Serverpod client:

```
client.auth.initialize();  
client.auth.initializeFirebaseSignIn();
```

FlutterFire configure fails or generates wrong config

Problem: `flutterfire configure` fails, or the generated `firebase_options.dart` has wrong project details.

Cause: The Firebase CLI is not installed, not logged in, or is pointing to the wrong project.

Resolution:

1. Install the Firebase CLI.
2. Log in: `firebase login`
3. Install FlutterFire CLI: `dart pub global activate flutterfire_cli`
4. Run `flutterfire configure` and select the correct project when prompted.

See the [FlutterFire CLI documentation](#) for more details.

FlutterFire configure fails for iOS or macOS apps

Problem: `flutterfire configure` registers the Android app successfully but fails when registering iOS or macOS apps, with an error about an invalid bundle identifier.

Cause: When a platform app does not yet exist in your Firebase project, FlutterFire creates it and derives the default bundle ID from your Flutter project folder name. Apple bundle identifiers must be valid reverse-DNS strings, so any character that is not allowed in a DNS label (most commonly an underscore) breaks auto-registration.

Resolution: Pass the bundle ID explicitly so FlutterFire does not derive it from the folder name:

```
flutterfire configure \  
  --ios-bundle-id="dev.serverpod.authproviders" \  
  --macos-bundle-id="dev.serverpod.authproviders"
```

Use a valid reverse-DNS identifier for your app. If you have already created the iOS and macOS apps manually in the Firebase Console, FlutterFire will reuse those entries instead of trying to create new ones.

Firestore UI auth actions not firing

Problem: The `AuthStateChangeAction<SignedIn>` or `AuthStateChangeAction<UserCreated>` actions on the `SignInScreen` never fire, so `controller.login()` is never called.

Cause: The action types don't match the authentication state changes from the providers you configured. For example, using `EmailAuthProvider` but only listening for one of the two states.

Resolution: Make sure you have actions for both `SignedIn` (returning users) and `UserCreated` (new users). See the [Customizing the UI](#) page for the complete code example.

Platform-specific Firebase SDK configuration issues

Problem: Firebase operations fail on a specific platform (iOS, Android, or Web) with errors about missing configuration.

Cause: The `flutterfire configure` command may not have configured all platforms, or platform-specific files were not placed correctly.

Resolution:

1. Re-run `flutterfire configure` and ensure you select all platforms you want to support.
2. For **iOS**: Verify that `GoogleService-Info.plist` is included in the Xcode project's Runner target.
3. For **Android**: Verify that `google-services.json` is in `android/app/`.
4. For **Web**: Verify that the Firebase config is loaded in `web/index.html` or via `firebase_options.dart`.

`Firebase.initializeApp()` throws `channel-error` on web in production

Problem: A Flutter web app works fine in `flutter run -d chrome` (debug), but the production build throws `channel-error` from `FirestoreHostApi.initializeCore` as soon as `Firebase.initializeApp()` runs. The browser console shows obfuscated frames like `aJs.$2` and `JX.fI` in `main.dart.js` with no clear message.

Cause: Flutter's incremental build cache under `.dart_tool/flutter_build/` can silently drop `firebase_core_web`'s plugin registration. The on-disk plugin registrant file still lists `FirestoreWeb.registerWith(registrar)`, but stale kernel `.dill` artifacts mean `dart2js` compiles `main.dart.js` without the wiring. `FirestorePlatform.instance` then stays as the default `MethodChannelFirestore`, and `Firebase.initializeApp()` goes through a pigeon channel with no

handler. The error sounds native, but it is purely a Flutter web build-cache issue. Debug mode (DDC) does not tree-shake or cache the same way as `dart2js` (release), so it hides the bug entirely.

Resolution: Run `flutter clean` before building so the registration is included:

```
flutter clean
flutter build web
```

For Serverpod Cloud projects, add `flutter clean` ahead of `flutter build web` in your `flutter_build` script (or whatever `scripts.pre_deploy` runs).

 TIP

To make minified stack traces readable while you debug an issue like this, build **locally** with `flutter build web --source-maps` and load that build (or the resulting `.map` file) in your browser's dev tools. Do **not** deploy the `.map` file to a public-facing host: it contains your full Dart source. If you need readable production stack traces continuously, upload source maps to an error-monitoring service (Sentry, Datadog) rather than serving them next to `main.dart.js`.